

API TESTING EDITION 2

Contents of Api Testing Edition 2

Module 1: Browser DevTools

- What is Browser DevTools?
- Network Tab
- XHR vs Fetch
- Request & Response Analysis
- Headers
- Payload
- Preview
- Response
- Timing
- Copy as cURL
- HAR File (Basics)

Module 2: Swagger / OpenAPI

- What is Swagger?
- Why Swagger is used
- Reading API Documentation
- Swagger UI
- API Specification
- Request Parameters
- Request Body
- Response Models
- Authentication in Swagger
- Testing APIs from Swagger

Module 3: Cookies & Sessions

- What is a Cookie?
- Types of Cookies
- Session Cookie
- Persistent Cookie
- Secure Cookie
- HttpOnly Cookie
- SameSite Cookie (Basics)
- What is a Session?
- Cookie vs Session
- Session ID

Module 4: Database Verification

- Why Database Verification is Required
- Manual Database Verification
- Insert Validation

- Update Validation
- Delete Validation
- Select Validation
- Basic SQL Queries
- SELECT
- WHERE
- ORDER BY
- COUNT
- Comparing API Response with Database

Module 5: Advanced Headers

- Content-Type
- Accept
- User-Agent
- Host
- Cache-Control
- Authorization
- Content-Length

Module 6: Advanced API Testing Scenarios

- Login API
- Registration API
- Create User API
- Update User API
- Delete User API
- Search API
- Filter API
- Pagination API
- File Upload API (Concept)
- Download API (Concept)

Module 7: Real-Time API Validation

- Request Validation
- Response Validation
- Business Validation
- Data Validation
- Response Time Validation
- Database Validation
- Error Validation
- Security Validation

Module 1: Browser DevTools

1. What is Browser DevTools?

Browser DevTools (Developer Tools) is a built-in browser feature that helps developers and testers inspect, debug, and analyze web applications. It allows us to inspect HTML, CSS, JavaScript, monitor network requests, view API calls, debug issues, and analyze application performance.

Real-Time Example

While testing a Login page, I use the Network tab in DevTools to verify whether the Login API is triggered, check the request payload, response, status code, and response time.

2. What is the Network Tab?

The Network Tab displays all network requests made by the browser, such as API calls, images, CSS, JavaScript files, fonts, and documents. It helps testers verify API communication between the frontend and backend.

What can we check?

- Request URL
- HTTP Method
- Status Code
- Request Headers
- Response Headers
- Request Payload
- Response Body
- Response Time

Real-Time Example

After clicking the Login button, I open the Network tab to ensure the Login API is called successfully and returns 200 OK.

3. What is the difference between XHR and Fetch?

XHR	Fetch
Uses XMLHttpRequest	Uses Fetch API
Older technology	Modern JavaScript API
More complex syntax	Simpler syntax
Still used in older applications	Preferred in modern applications

In DevTools, you may see XHR or Fetch/XHR filters. Both help you view API requests.

4. What is Request & Response Analysis?

Request and Response Analysis means examining the data sent to the server and the data returned by the server to ensure they match the expected behavior.

Verify:

- URL
- HTTP Method
- Headers
- Request Body
- Authentication

Verify:

- Status Code
- Response Body
- Headers
- Response Time
- Error Messages

Example

Create User API:

Request

```
{  
  "name": "Shankar",  
  "email": "shankar@test.com"  
}
```

Response

```
{  
  "id": 101,  
  "name": "Shankar",  
  "email": "shankar@test.com"  
}
```

5. What are Headers?

Headers are key-value pairs sent in both the request and response. They provide additional information about the request or response.

Common Request Headers

- Authorization
- Content-Type
- Accept
- User-Agent

Common Response Headers

- Content-Type
- Content-Length
- Server
- Cache-Control

Example

Content-Type: application/json

Authorization: Bearer eyJhbGci...

6. What is Payload?

A payload is the actual data sent to the server in the request body. It is mainly used with POST, PUT, and PATCH requests.

Example

```
{  
  "username": "shankar",  
  "password": "Password@123"  
}
```

Real-Time Example

When creating a new trip in a Transportation Management System (TMS), the trip details such as driver ID, vehicle ID, and route are sent as the payload.

7. What is Preview?

The Preview tab in DevTools displays the API response in a user-friendly, formatted view. It makes it easier to understand nested JSON data.

Example

Instead of viewing raw JSON text, Preview shows expandable objects and arrays.

8. What is Response?

The Response tab displays the raw data returned by the server after processing the request.

Things to Verify

- Response data
- Error messages
- JSON structure
- Business data
- Status information

Example

```
{  
  "message": "Login Successful",  
  "token": "eyJhbGciOi..."  
}
```

9. What is Timing?

The Timing tab shows how long each stage of the network request takes, helping identify performance issues.

It includes:

- DNS Lookup
- Initial Connection
- SSL Handshake
- Request Sent
- Waiting (Server Processing)
- Content Download

Real-Time Example

If a Login API takes 6 seconds instead of the expected 2 seconds, the Timing tab helps identify where the delay occurs.

10. What is "Copy as cURL"?

Copy as cURL copies the selected network request as a cURL command. This allows the same request to be executed from a terminal or imported into tools like Postman.

Why is it useful?

- Reproduce API requests.
- Share requests with developers.
- Import requests into Postman for testing.
- Troubleshoot API issues.

Example

```
curl -X POST https://api.company.com/login \  
-H "Content-Type: application/json" \  
-d '{"username":"'shankar','password":"'Password@123"}'
```

11. What is a HAR File?

A HAR (HTTP Archive) File is a JSON-based file that records all network activity during a browser session. It contains detailed information about every request and response.

It includes:

- Request URLs
- HTTP Methods
- Headers
- Payloads
- Responses
- Status Codes
- Timings

Why is it used?

- Debug performance issues.
- Share network logs with developers.
- Analyze failed API requests.
- Troubleshoot browser-related problems.

Real-Time Example

If users report that the application is slow, QA can export a HAR file from the Network tab and share it with developers to analyze the

network traffic and identify bottlenecks.

Module 2: Swagger / OpenAPI

1. What is Swagger?

Swagger is an open-source framework used to design, document, and test REST APIs. It provides an interactive interface where developers and testers can view API endpoints, request parameters, request bodies, response models, and test APIs directly from the browser.

Real-Time Example

In my project, if I needed to test the Create User API, I could open Swagger, view the endpoint, understand the required request body, and execute the API without writing any code.

2. Why is Swagger used?

Swagger is used to simplify API development, documentation, and testing.

Advantages

- Provides complete API documentation.
- Displays all available API endpoints.
- Allows testers to execute APIs directly.
- Shows request and response formats.
- Helps developers and testers understand API behavior.
- Reduces dependency on developers for API details.

Real-Time Example

Instead of asking a developer for the Login API details, I can open Swagger and find the endpoint, HTTP method, request body, and expected responses.

3. What is API Documentation?

API Documentation is a document that explains how an API works. It contains all the information required to use and test an API.

It usually includes:

- Base URL
- Endpoints
- HTTP Methods
- Request Parameters
- Request Body
- Authentication
- Response Models
- Status Codes
- Error Messages

Real-Time Example

Before testing a Trip API, I first read the API documentation to understand the required fields and expected responses.

4. What is Swagger UI?

Swagger UI is a web-based interface that displays API documentation in a user-friendly format. It allows users to execute API requests directly from the browser.

Features

- View all API endpoints.
- Enter request parameters.
- Provide a request body.
- Add authentication.
- Execute APIs.
- View request and response.

Real-Time Example

I can click the POST /login endpoint in Swagger UI, enter credentials, click Execute, and immediately see the API response.

5. What is an API Specification?

An API Specification is a standard document that defines how an API should behave. It describes the endpoints, request format, response format, authentication, and error responses.

Swagger commonly uses the OpenAPI Specification (OAS) to describe REST APIs.

It contains:

- API name
- Base URL
- Endpoints
- HTTP methods
- Parameters
- Request body
- Response models
- Status codes
- Security requirements

Real-Time Example

Developers follow the API specification while developing APIs, and testers use the same specification to verify whether the API behaves as expected.

6. What are Request Parameters?

Request Parameters are values sent with an API request to provide input to the server.

Types

- Path Parameters
- Query Parameters
- Header Parameters

- Cookie Parameters

Example

Path Parameter

GET /users/101

Here, 101 is the Path Parameter.

Query Parameter

GET /users?city=Bangalore

Here, city=Bangalore is the Query Parameter.

Real-Time Example

To retrieve trip details for Trip ID 1001, I send the Trip ID as a Path Parameter.

7. What is a Request Body?

A Request Body contains the data sent to the server when creating or updating a resource. It is commonly used with POST, PUT, and PATCH requests.

Example

```
{  
  "name": "Shankar",  
  "email": "shankar@test.com",  
  "role": "QA Engineer"  
}
```

Real-Time Example

When creating a new user, the user's details are sent in the Request Body.

8. What are Response Models?

A Response Model defines the structure of the data returned by an API. It helps testers understand what fields should be present in the response.

Example

```
{  
  "id": 101,  
  "name": "Shankar",  
  "email": "shankar@test.com",  
  "status": "Active"  
}
```

What do we validate?

- Field names
- Data types
- Mandatory fields
- Business data
- Response structure

Real-Time Example

After creating a trip, I verify that the response includes the generated Trip ID, Driver ID, Status, and other expected fields.

9. What is Authentication in Swagger?

Authentication in Swagger allows users to access secured APIs by providing valid credentials or tokens.

Common Authentication Types

- Basic Authentication
- Bearer Token
- API Key

- OAuth 2.0

Example (Bearer Token)

Authorization: Bearer eyJhbGciOiJIUzI1Ni...

Real-Time Example

I first execute the Login API to obtain a Bearer Token, then click Authorize in Swagger UI, paste the token, and access protected APIs.

10. How do you test APIs from Swagger?

The steps are:

1. Open Swagger UI.
2. Select the required API endpoint.
3. Click Try it out.
4. Enter request parameters or request body.
5. Add authentication if required.
6. Click Execute.
7. Verify:
 - Status Code
 - Response Body
 - Response Headers
 - Error Messages
 - Business Data

Real-Time Example

To test the Create User API:

- Open the POST `/users` endpoint.
- Click Try it out.
- Enter the user details.
- Click Execute.
- Verify that the API returns 201 Created and the response contains the generated User ID.

Module 3: Cookies & Sessions

1. What is a Cookie?

A **Cookie** is a small piece of data stored in the user's web browser by a website. It helps the website remember user information such as login status, preferences, and session details.

Why are Cookies used?

- Store login information
- Remember user preferences
- Maintain user sessions
- Track user activity
- Improve user experience

Real-Time Example

When you log in to Gmail and select "**Remember Me**", the browser stores a cookie so you don't have to log in again every time you visit.

2. What are the Types of Cookies?

The common types of cookies are:

- Session Cookie
- Persistent Cookie
- Secure Cookie
- HttpOnly Cookie
- SameSite Cookie

Each type has a different purpose related to security and user experience.

3. What is a Session Cookie?

A **Session Cookie** is a temporary cookie that exists only while the browser is open. It is automatically deleted when the browser is closed.

Uses

- Maintain login session
- Store temporary user information
- Track user actions during a session

Real-Time Example

You log in to an online banking application. If you close the browser without logging out, the session cookie is deleted, and you'll need to log in again.

4. What is a Persistent Cookie?

A **Persistent Cookie** remains stored in the browser even after the browser is closed. It stays until its expiration date or until the user deletes it.

Uses

- Remember login information
- Save language preferences
- Save theme settings
- Remember shopping cart items

Real-Time Example

When you select "**Keep me signed in**" on Amazon, a persistent cookie allows you to remain logged in even after restarting your browser.

5. What is a Secure Cookie?

A **Secure Cookie** is transmitted only over **HTTPS** connections. It helps protect sensitive information from being intercepted over unsecured networks.

Uses

- Protect authentication data
- Enhance application security
- Prevent data exposure over HTTP

Real-Time Example

In online banking applications, authentication cookies are marked as **Secure** so they are only sent over encrypted HTTPS connections.

6. What is an HttpOnly Cookie?

An **HttpOnly Cookie** cannot be accessed through JavaScript. It can only be sent between the browser and the server.

Advantages

- Protects against Cross-Site Scripting (XSS) attacks
- Prevents JavaScript from stealing session cookies
- Improves application security

Real-Time Example

A banking application stores the session cookie as **HttpOnly**, preventing malicious scripts from accessing it.

7. What is a SameSite Cookie?

A **SameSite Cookie** controls whether cookies are sent with cross-site requests. It helps protect applications from **Cross-Site Request Forgery (CSRF)** attacks.

SameSite Modes

- **Strict** – Cookies are sent only for requests from the same website.
- **Lax** – Cookie is sent for some cross-site navigation requests.
- **None** – Cookies are sent with all requests but must also be marked as **Secure**.

Real-Time Example

A banking website uses **SameSite=Strict** to prevent other websites from sending authenticated requests on behalf of the user.

8. What is a Session?

A **Session** is a server-side mechanism used to store user information while the user is interacting with the application.

Unlike cookies, session data is stored on the **server**, not in the browser.

Uses

- Maintain login state
- Store user information
- Track user activities
- Manage authentication

Real-Time Example

After logging into an e-commerce application, the server creates a session to remember your identity until you log out or the session expires.

9. Difference Between Cookie and Session

Cookie	Session
Stored in the browser	Stored on the server
Less secure	More secure

Can persist after browser closure (Persistent Cookie)	Usually ends after logout or timeout
Limited storage (around 4 KB)	Larger storage capacity
Sent with every request to the server	Server identifies the user using a Session ID

Real-Time Example

When you log in:

- The **Session** containing your login information is created on the server.
- The browser stores a **Cookie** containing the Session ID.
- On each request, the browser sends the Session ID, allowing the server to identify your session.

10. What is a Session ID?

A **Session ID** is a unique identifier generated by the server when a user logs in. It links the browser to the user's session stored on the server.

The Session ID is usually stored in a cookie.

Uses

- Identify the user's session
- Maintain authentication
- Track user requests
- Retrieve session data from the server

Real-Time Example

1. User logs into a Transportation Management System (TMS).
2. The server creates a session.
3. The server generates a Session ID such as:

SID=ab12cd34ef56gh78

4. The browser stores this Session ID in a cookie.
5. Every subsequent request includes the Session ID.
6. The server uses it to recognize the logged-in user and authorize access

Module 4: Database Verification

1. Why is Database Verification Required?

Database Verification is the process of validating that the data stored in the database is correct after an API or application performs an operation.

It ensures that the application has correctly inserted, updated, deleted, or retrieved data from the database.

Why is it important?

- Verify data integrity
- Ensure correct data is stored
- Validate business rules
- Detect backend defects
- Confirm API functionality

Real-Time Example

Suppose you create a new Trip in a Transportation Management System (TMS).

After receiving 201 Created, you execute an SQL query to verify that the Trip details are actually stored in the database.

2. What is Manual Database Verification?

Manual Database Verification is the process of checking the database manually using SQL queries to verify whether the data matches the application's expected results.

QA engineers use SQL tools such as:

- MySQL Workbench
- SQL Server Management Studio (SSMS)
- Oracle SQL Developer
- pgAdmin

Steps

1. Perform an operation in the application or API.
2. Connect to the database.
3. Execute SQL queries.
4. Compare the database values with the application or API response.

Real-Time Example

Create a new Driver using an API.

Execute:

SELECT *

FROM Drivers

WHERE DriverID = 1001;

Verify that the driver's information matches the API request.

3. What is Insert Validation?

Insert Validation verifies that a new record has been successfully inserted into the database after an operation.

What do we verify?

- Record exists
- Correct values are stored
- No duplicate records
- Mandatory fields are populated

Example

API Request:

```
{  
  "name": "Shankar",  
  "email": "shankar@test.com"  
}
```

SQL Query

SELECT *

FROM Users

WHERE Email='shankar@test.com';

Expected Result

A new user record should exist.

4. What is Update Validation?

Update Validation verifies that an existing record has been modified correctly after an update operation.

What do we verify?

- Correct record updated
- Updated values are stored
- Other fields remain unchanged

Example

Before Update

Driver Name = Ravi

API Request

```
{  
  "driverName": "Rahul"  
}
```

SQL Query

```
SELECT DriverName  
FROM Drivers  
WHERE DriverID=101;
```

Expected Result

Driver Name = Rahul

5. What is Delete Validation?

Delete Validation verifies that a record has been deleted or marked inactive after a delete operation.

What do we verify?

- Record no longer exists
- Soft delete flag updated (if applicable)
- API returns the correct response

Example

Delete User API

```
DELETE /users/101
```

SQL Query

```
SELECT *  
FROM Users  
WHERE UserID=101;
```

Expected Result

No record found.

OR

Status = Inactive

(if soft delete is used)

6. What is Select Validation?

Select Validation verifies that the data returned by the application or API matches the data stored in the database.

What do we verify?

- Correct records returned
- Correct field values
- Correct sorting
- Correct filtering

Example

API Response

```
{  
  "name": "Shankar"  
}
```

SQL Query

SELECT Name

FROM Users

WHERE UserID=101;

Both values should match.

7. What is the SELECT Statement?

The SELECT statement is used to retrieve data from a database table.

Syntax

```
SELECT *  
FROM Users;
```

Example

Retrieve all users.

```
SELECT *  
FROM Users;
```

8. What is the WHERE Clause?

The WHERE clause is used to filter records based on a specified condition.

Syntax

```
SELECT *  
FROM Users  
WHERE UserID=101;
```

Example

Retrieve a specific user.

```
SELECT *
```

```
FROM Users
```

```
WHERE Email='shankar@test.com';
```

9. What is ORDER BY?

The ORDER BY clause is used to sort query results in ascending or descending order.

Ascending

```
SELECT *
```

```
FROM Users
```

```
ORDER BY UserID ASC;
```

Descending

```
SELECT *
```

```
FROM Users
```

```
ORDER BY UserID DESC;
```

Real-Time Example

Display the latest trips first.

```
SELECT *
```

```
FROM Trips
```

```
ORDER BY CreatedDate DESC;
```

10. What is COUNT()?

The COUNT() function returns the total number of records in a table.

Example

```
SELECT COUNT(*)
```

```
FROM Users;
```

Result

250 Users

Example with WHERE

```
SELECT COUNT(*)
```

```
FROM Trips
```

```
WHERE Status='Completed';
```

This returns the number of completed trips.

11) Comparing API Response with Database

Comparing the API response with the database means verifying that the data returned by the API matches the data stored in the database. This helps ensure that the API has correctly inserted, updated, deleted, or retrieved the data.

Example:

If I create a new user using a Create User API, the API returns:

```
{  
  "userId": 101,  
  "name": "Shankar",  
  "email": "shankar@test.com"  
}
```

Then I execute an SQL query:

```
SELECT * FROM Users
```

WHERE UserID = 101;

I compare the API response with the database record to ensure that the User ID, Name, Email, and other details match. If there is any mismatch, I report it as a defect.

Module 5: Advanced Headers

1. Content-Type

The **Content-Type** header specifies the format of the data being sent in the request or returned in the response.

Common Values

- application/json
- application/xml
- text/html
- multipart/form-data

Example

Content-Type: application/json

Real-Time Example

When sending a POST request with JSON data, I use:

Content-Type: application/json

This tells the server that the request body is in JSON format.

2. Accept

The **Accept** header tells the server the format of the response that the client expects.

Common Values

- application/json

- application/xml

Example

Accept: application/json

Real-Time Example

If I want the API response in JSON format, I set:

Accept: application/json

3. User-Agent

The **User-Agent** header identifies the client (browser, mobile app, or tool) that is sending the request.

Example

User-Agent: PostmanRuntime/7.39.0

or

User-Agent: Mozilla/5.0

Real-Time Example

When sending a request from Postman, the User-Agent indicates that the request originated from Postman.

4. Host

The **Host** header specifies the server or domain name to which the request is sent.

Example

Host: api.company.com

Real-Time Example

If the API URL is:

`https://api.company.com/users`

The Host header is:

Host: `api.company.com`

5. Cache-Control

The **Cache-Control** header controls how responses are cached by browsers or intermediate servers.

Common Values

- no-cache
- no-store
- max-age=3600

Example

Cache-Control: no-cache

Real-Time Example

For sensitive APIs like Login or Payment, Cache-Control: no-cache ensures the browser fetches fresh data instead of using cached data.

6. Authorization

The **Authorization** header is used to send authentication credentials to access protected APIs.

Example (Bearer Token)

Authorization: Bearer `eyJhbGciOiJIUzI1Ni...`

Example (Basic Authentication)

Authorization: Basic `dXNlcjpwYXNzd29yZA==`

Real-Time Example

After logging in, I receive a Bearer Token. I include it in the Authorization header when calling protected APIs like **Create Trip** or **Update User**.

7. Content-Length

The **Content-Length** header indicates the size of the request or response body in bytes.

Example

Content-Length: 245

Real-Time Example

If the request body contains 245 bytes of data, the Content-Length header will be:

Content-Length: 245

The server uses this value to know how much data to read.

Module 6: Advanced API Testing Scenarios

1. How do you test a Login API?

When testing a Login API, I perform the following validations:

- Verify the HTTP method is **POST**.
- Send valid username and password.
- Verify the status code is **200 OK**.
- Validate that an authentication token is generated.
- Verify the response body contains user details.
- Perform negative testing using invalid credentials.
- Verify appropriate error messages for invalid login.

- Check response time.

Example

Request

```
{  
  "username": "shankar",  
  "password": "Test@123"  
}
```

Expected Result

- Status Code: **200 OK**
 - Bearer Token generated
 - User details returned
-

2. How do you test a Registration API?

When testing a Registration API, I verify:

- Mandatory fields
- Valid user details
- Duplicate email or username
- Invalid email format
- Password validation
- Status code (**201 Created**)
- Success message
- User record created in the database

Example

Request

```
{  
  "name": "Shankar",
```

```
"email": "shankar@test.com",  
"password": "Test@123"  
}
```

Expected Result

- User created successfully
 - Status Code: **201 Created**
-

3. How do you test a Create User API?

I perform the following validations:

- Verify the HTTP method is **POST**.
- Send valid request data.
- Verify the status code (**201 Created**).
- Validate the response body.
- Verify User ID generation.
- Verify data is stored in the database.
- Perform negative testing with invalid or missing data.

Example

Expected Response

```
{  
"id": 101,  
"name": "Shankar"  
}
```

4. How do you test an Update User API?

When testing an Update API:

- Verify the HTTP method is **PUT** or **PATCH**.
- Update an existing user's information.
- Verify the status code (**200 OK**).
- Verify only the intended fields are updated.
- Verify changes in the database.
- Perform negative testing with invalid User IDs.

Example

Before Update

Name = Shankar

Request

```
{  
  "name": "Shankar S"  
}
```

Expected Result

Name = Shankar S

5. How do you test a Delete User API?

When testing a Delete API:

- Verify the HTTP method is **DELETE**.
 - Delete an existing user.
 - Verify the status code (**200 OK** or **204 No Content**).
 - Verify the record is deleted or marked inactive.
 - Verify retrieving the deleted user returns **404 Not Found**.
-

6. How do you test a Search API?

A Search API retrieves records based on search criteria.

Validations

- Search with valid data.
- Search with invalid data.
- Partial search.
- Case-sensitive and case-insensitive search.
- Empty search.
- Response time.
- Correct records returned.

Example

GET /users?name=Shankar

Expected Result

Only users named **Shankar** should be returned.

7. How do you test a Filter API?

A Filter API returns records based on selected conditions.

Validations

- Single filter.
- Multiple filters.
- Invalid filter values.
- Empty filter.
- Combination of filters.
- Response time.

Example

GET /orders?status=Delivered

Expected Result

Only **Delivered** orders should be returned.

8. How do you test a Pagination API?

Pagination divides a large amount of data into smaller pages.

Validations

- Verify page number.
- Verify page size.
- Verify total records.
- Verify the first page.
- Verify the last page.
- Verify invalid page number.
- Verify duplicate records are not displayed.

Example

GET /users?page=1&limit=10

Expected Result

The API should return only **10 users** from page **1**.

9. How do you test a File Upload API? (Concept)

A File Upload API allows users to upload files such as images, PDFs, or documents.

Validations

- Upload a valid file.
- Upload an invalid file type.
- Upload a large file.
- Upload an empty file.
- Verify file size restrictions.
- Verify duplicate uploads.
- Verify the uploaded file is stored successfully.

Example

POST /upload

Upload

resume.pdf

Expected Result

- Status Code: **200 OK**
 - File uploaded successfully.
-

10. How do you test a Download API? (Concept)

A Download API allows users to download files from the server.

Validations

- Download a valid file.
- Verify file name.
- Verify file size.
- Verify file format.
- Download with invalid ID.
- Download without authorization.
- Verify downloaded content.

Example

GET /download/Invoice101.pdf

Expected Result

Invoice **101.pdf** should be downloaded successfully.

Module 7: Real-Time API Validation

1. What is Request Validation?

Request Validation is the process of verifying that the API request sent to the server is correct before processing.

What do we validate?

- HTTP Method (GET, POST, PUT, DELETE)
- Request URL
- Headers
- Authentication
- Request Body
- Mandatory Fields
- Data Types

Example

Create User API

```
{  
  "name": "Shankar",  
  "email": "shankar@test.com"  
}
```

Verify that all mandatory fields are present and correctly formatted.

2. What is Response Validation?

Response Validation is the process of verifying that the API returns the expected response after processing the request.

What do we validate?

- Status Code
- Response Body
- Response Headers
- Data Types

- Success/Error Message
- Response Time

Example

Expected Response

```
{  
  "id":101,  
  "name":"Shankar",  
  "status":"Active"  
}
```

Verify that the response matches the expected result.

3. What is Business Validation?

Business Validation ensures that the API follows the application's business rules and requirements.

Example

In an e-commerce application:

- A user should not be able to place an order if the product is out of stock.

Expected Result:

The API should return an appropriate error message instead of creating the order.

4. What is Data Validation?

Data Validation ensures that the data returned by the API is correct, complete, and matches the expected values.

What do we validate?

- Field Values

- Data Types
- Mandatory Fields
- Null Values
- Duplicate Data

Example

API Response

```
{  
  "name": "Shankar",  
  "age": 28  
}
```

Verify that the name and age values are correct.

5. What is Response Time Validation?

Response Time Validation checks whether the API responds within the expected time limit.

Example

Expected Response Time:

- Login API → Less than **2 seconds**
- Search API → Less than **3 seconds**

If the API takes longer than the expected time, it may indicate a performance issue.

6. What is Database Validation?

Database Validation verifies that the data stored in the database matches the API request and response.

Example

After executing a **Create User API**, run an SQL query:

```
SELECT *  
FROM Users  
WHERE UserID = 101;
```

Verify that the User ID, Name, and Email match the API response.

7. What is Error Validation?

Error Validation ensures that the API returns appropriate error messages and status codes when invalid requests are sent.

Example

Login API

Invalid Password

Expected Result

- Status Code: **401 Unauthorized**
- Error Message:

```
{  
"message": "Invalid Username or Password"  
}
```

8. What is Security Validation?

Security Validation ensures that only authorized users can access protected APIs and that sensitive data is secure.

What do we validate?

- Authentication
- Authorization
- HTTPS
- Token Validation

- Access Permissions

Example

If a user accesses a protected API without a valid Bearer Token, the API should return:

401 Unauthorized